Numerical-Text Surrogate Modelling and Multi-Objective Optimisation of Compact Patch Antennas

Student name: Ms. Rathi Devadhas

Student ID: 1737089

Programme: Master of Applied Artificial Intelligence

Course: Applied AI Research Workshop, Module 2

Code repository: https://github.com/rathiabeth-sudo/902---Module-2

Interactive demo: https://rathiabeth-sudo.github.io/902---Module-2/demo/

Abstract

Designing a compact patch antenna usually means running one full-wave simulation after another, and each run can take minutes to hours. In this project I built the numerical half of a two-module framework: models that predict antenna performance from a handful of dimensions, a short text summary for every prediction, a test of whether these numbers add anything to the image features from Module 1, and a multi-objective search for 5G NR n78 designs. Ansys HFSS was not available, so everything was trained and tested on two public simulation datasets. A random forest predicted resonance with $R^2 = 0.89$ and bandwidth with $R^2 = 0.83$, but matching depth only with $R^2 = 0.54$. Combining numbers with image features did not help (macro-F1 0.737 against 0.775 for images alone). The 100 designs proposed by the optimiser are predictions that still need full-wave checks. An interactive demo of the models is available online at https://rathiabeth-sudo.github.io/902---Module-2/demo/.

1. Introduction

Patch antennas for 5G and future 6G devices have to be small, well matched and tuned to the right band. Cutting slots into the patch or etching a defected ground structure (DGS) gives

the designer more freedom, but it also makes the response sensitive to tiny changes in geometry, so a parameter sweep in a full-wave solver such as HFSS can run for days [4]. The wider research framework tackles this in two parts. Module 1, in its own repository, is a Vision Transformer [1] that classifies images of antenna layouts. Module 2, which this report covers, works with numbers instead of pictures.

My starting question was simple: if I only know an antenna's dimensions, how well can I predict what the simulator would say? From there I wanted to find out whether those numbers improve on the image features from Module 1, and whether a fast surrogate is good enough to steer an optimiser towards sensible designs for the n78 band (3.3–3.8 GHz). I also set myself one rule for the whole project. Every number and every sentence the system produces has to be traceable to a model output or a simulated value. A tool that sounds confident but invents results would be worse than useless to an antenna engineer.

2. Data and Preparation

I searched Hugging Face, Kaggle, Mendeley Data, IEEE DataPort, GitHub and Zenodo for usable data. Of eleven candidates, only two met my requirements: clear geometric inputs, simulated electromagnetic outputs, a licence that allows reuse, and enough designs to keep some aside for testing. I could not find any public HFSS sweep of hybrid-slotted DGS antennas, and that gap shapes everything that follows.

The first dataset holds 2,051 coax-fed rectangular patches simulated in openEMS [10]. Each design is described by its length, width and feed offset, and comes with a 1,000-point S11 curve from 1 to 10 GHz [2]. The second is the Mendeley Antenna Parameters Dataset: 103 dual-slot patch designs, each simulated at 16 frequencies, with S11, gain and efficiency [3].

Neither dataset was ready to use as it came. For the patch data I reduced each S11 curve to four targets: the first matched resonance (the lowest-frequency dip below −10 dB), the minimum S11, the S11 at that resonance and the −10 dB bandwidth. I chose the first matched resonance on purpose, because the deepest dip is often a higher-order mode; a 22.5 mm patch, for example, has its deepest dip at 9.6 GHz. The Mendeley data needed more work. One efficiency value was a truncated text string, and 245 of the 1,648 rows were physically impossible, with efficiencies above 1 or gain higher than directivity. I flagged these rows with a written rule instead of tuning a filter until the numbers looked good. They stayed in for S11, which they do not affect, and were left out of the gain and efficiency targets.

To keep the comparison with Module 1 fair, I reused its locked split: 1,220 training, 262 validation and 262 test designs, plus the 307 widest patches (W/L above 1.95) held back as an out-of-distribution (OOD) set. For the Mendeley data I used five-fold cross-validation grouped by design, so the 16 frequencies of one antenna never ended up on both sides of a split.

3. Methods

For each target I trained four models with scikit-learn [7]: a mean predictor as a floor, ridge regression, histogram gradient boosting and a random forest of 400 trees [6]. The model with the lowest validation error was kept, and the test set was scored only once, at the very end. I expected the tree models to win because the physics is non-linear. Resonance depends mostly on length, while matching depends on where the feed sits relative to the edge of the patch. The inputs were length, width, feed offset, the W/L ratio and a normalised feed position.

The text part is deliberately low-tech. Instead of a language model, a template reads the predictions and the simulated reference for the same design, places the resonance in a 3GPP band [5], applies the matching rule and writes one paragraph. It cannot make up a number because it

only copies computed values, and every summary ends by saying that the design still needs full-wave verification.

For the fusion experiment I joined the 384-dimensional embedding exported by Module 1 with the five numerical features and trained a small neural network to predict a three-level matching class (Poor, Acceptable or High). I compared it against the same network given only the embedding or only the numbers. Before any results existed, I registered hypothesis H6: fusion would beat the image-only model.

Finally, I used NSGA-II [8] from the pymoo library [9] to search over length, W/L and feed position. The search tried to deepen the predicted match, widen the bandwidth and shrink the patch area, while keeping the predicted resonance inside n78. It ran with 100 candidates for 80 generations. I then compared each final design with its nearest simulated neighbour to see how far the surrogate was being pushed beyond its data.

4. Results

Table 1 summarises the patch surrogate. The random forest was selected for all four targets and beat the mean baseline every time. Resonance was predicted well, with an average error of 0.35 GHz, and so was bandwidth, at 18 MHz. Matching depth was much harder (R^2 = 0.54). That makes physical sense. Resonance shifts smoothly as the patch gets longer, but the depth of the S11 dip depends on how close the input impedance gets to 50 Ω, and a small move of the feed can change it by many decibels.

Table 1. *Patch surrogate (random forest) on the locked test split (262 designs) and the OOD hold-out (307 designs).*

Target	Test R^2	Test MAE	Mean-baseline MAE	OOD R^2
First matched resonance	0.89	0.35 GHz	1.82 GHz	0.44
−10 dB bandwidth	0.83	18.1 MHz	55.4 MHz	−3.72
Minimum S11	0.54	4.8 dB	7.4 dB	0.13
S11 at resonance	0.57	3.3 dB	5.6 dB	−1.54

The OOD results were the most sobering part of the project. On the wide patches the model had never seen, R^2 for resonance dropped to 0.44, and for bandwidth and S11 at resonance it fell below zero. Tree models simply cannot extrapolate past their training range. This changed how I set up the rest of the work: I limited the optimiser to the training ranges and checked every proposal against real simulated designs.

On the slotted Mendeley antenna the forests reached $R^2 = 0.86$ for S11, 0.99 for gain and 0.97 for radiation efficiency. Adding the ten geometry variables roughly halved the error of a model that only knew the frequency (0.53 dB against 1.09 dB for S11). I would not read too much into the very high gain score, though. The frequency-only model already reaches $R^2 = 0.98$, and because the dataset changes one dimension at a time around a single nominal design, the test folds still contain near-identical antennas.

Fusion did not work out the way I had hoped. The combined model scored a test macro-F1 of 0.737, below the image-only model at 0.775 and above the numbers-only model at 0.716, so H6 is not supported. The more interesting finding was the image-only score itself. Module 1's own ViT classifier reached only 0.581, so a simple network on the same embeddings did far better. This supports Module 1's diagnosis that its classifier head underfitted while the representation underneath had already learned the geometry. Adding five numbers to 384 image dimensions mostly gave a small training set more redundant inputs to deal with.

NSGA-II returned 100 proposals covering 299–2,754 mm², with predicted bandwidths of 51–103 MHz. They fell into two groups that make physical sense: 71 compact patches about 19–23 mm long that use the fundamental mode, and 29 larger patches about 39–41 mm long whose first matched resonance is a higher-order mode, trading size for a deeper match. Their median distance to the nearest simulated design was only 0.08 standard deviations, so they are interpolations rather than discoveries. The widest predicted bandwidth beats the best simulated n78 design (99 MHz) by less than the model's own bandwidth error, so I cannot claim that any proposal is a real improvement. For a dense sweep of a simple patch, that is what I expected. The approach should pay off when the design space is much larger than what has been simulated, as it will be with a DGS sweep.

The summaries did what I wanted. For test design P1768 the generator reported a predicted resonance of 3.57 GHz, inside n78, next to the simulated 3.54 GHz. Putting the prediction and the reference side by side lets a reader see the error straight away. The weakness is that a template can restate numbers but cannot explain why a design behaves the way it does.

5. Interactive Demo

To make the models easier to explore, I built a browser demo, available at https://rathiabeth-sudo.github.io/902---Module-2/demo/. It uses the same two datasets and the same trained random forests as this report. In the design explorer you move sliders for length, W/L and feed position and see the predicted resonance, S11 and bandwidth, the matching class, a check against the n78, n77 or n79 band, a generated engineering summary and the simulated S11 curve of the nearest real design. Other tabs compare predictions with openEMS on the 262 test designs, plot the 100 NSGA-II proposals, show the Mendeley S11 and gain curves and list the

model scores. Because the page has no server behind it, I pre-computed the forest's predictions on a grid of 84,525 designs. The script that builds this grid is in the repository.

6. Limitations and Responsible Use

The biggest limitation is the topology gap. The patch data has no slots or DGS, and the Mendeley data varies one dimension at a time, so these surrogates will not transfer directly to hybrid-slotted DGS antennas. All targets come from openEMS or the Mendeley authors' solver rather than HFSS, and nothing was fabricated or measured. The −20 dB boundary for the High class is my own assumption and needs confirmation from an antenna engineer. The fusion test also used only one embedding run and one seed.

Both datasets are public under MIT and CC BY 4.0 licences, contain no personal data, and are cited as their terms require. A script downloads the raw files and checks their checksums, the settings live in one configuration file with fixed seeds, and ten automated tests cover the target extraction, the split, the cleaning rule and the optimiser constraints. All ten pass. Every prediction, summary and proposal is labelled as a screening result that needs full-wave verification.

7. Conclusion

Module 2 shows that a few antenna dimensions are enough to predict simulated resonance and bandwidth well, and matching depth only moderately. On a slotted patch, adding the geometry roughly halves the error of a frequency-only model. Combining numbers with ViT embeddings did not beat the embeddings alone, but it did reveal that Module 1's representation is stronger than its own classifier suggested. The optimiser produced sensible n78 candidates, all close to existing designs and all unverified. The main lesson for the full HFSS study is that a surrogate is only as good as its coverage. The DGS sweep should be space-filling rather than

one-at-a-time, the surrogate should report its own uncertainty, and every optimised design should be treated as a hypothesis to test in the simulator. My next steps are to generate that sweep with PyAEDT, add uncertainty estimates, and feed re-simulated proposals back into training.

Data, Code and Demo Availability

Code, data, tests and results: https://github.com/rathiabeth-sudo/902---Module-2

Interactive demo: https://rathiabeth-sudo.github.io/902---Module-2/demo/

Module 1 (Vision Transformer): https://github.com/rathiabeth-sudo/901---module-1

References

[1] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. ICLR, 2021.

[2] B. LaBash et al., "Improving generative inverse design of rectangular patch antennas with test time optimization," arXiv:2505.18188, 2025. Dataset: huggingface.co/datasets/becklabash/rectangular-patch-antenna-freq-response (MIT).

[3] M. F. H. Mihad, M. M. Islam and F. H. Sojib, "Antenna Parameters Dataset," Mendeley Data, v2, 2024, doi:10.17632/3gxr2vvd9n.2 (CC BY 4.0).

[4] C. A. Balanis, Antenna Theory: Analysis and Design, 4th ed. Hoboken, NJ, USA: Wiley, 2016.

[5] 3GPP, "NR; User Equipment (UE) radio transmission and reception; Part 1: Range 1 Standalone," TS 38.101-1.

[6] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[8] K. Deb, A. Pratap, S. Agarwal and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” IEEE Trans. Evol. Comput., vol. 6, no. 2, pp. 182–197, 2002.

[9] J. Blank and K. Deb, “pymoo: Multi-objective optimization in Python,” IEEE Access, vol. 8, pp. 89497–89509, 2020.

[10] T. Liebig et al., openEMS: a free and open electromagnetic field solver. [Online]. Available: https://openems.de